Vikram Thirumaran

October 23rd, 2025

CS 471 / Introduction to Artificial Intelligence

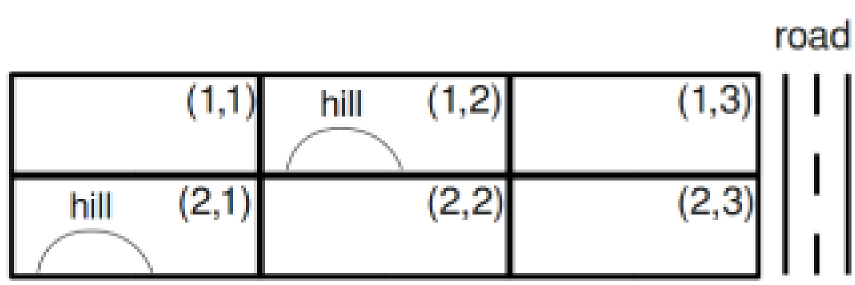
Fall 2025/ Prof. Thanh Nguyen

Written Assignment 2

Instruction: Remember the collaboration guidelines set forth in class: you may meet to discuss problems with classmates, but you may not take any written notes (or electronic notes, or photos, etc.) away from the meeting. Please complete the write-ups individually. (This applies to BOTH undergraduates and graduate students.) Your answers must be typewritten, except for figures or diagrams, which may be hand-drawn.

Q1. Campus Layout (32 points, 4 points each question)

You are asked to determine the layout of a new, small college. The campus will have four structures: an administration structure (A), a bus stop (B), a classroom (C), and a dormitory (D). Each structure (including the bus stop) must be placed somewhere on the grid shown below.



The layout must satisfy the following constraints:

(i) The bus stop (B) must be adjacent to the road.

(ii) The administration structure (A) and the classroom (C) must both be adjacent to the bus stop (B).

(iii) The classroom (C) must be adjacent to the dormitory (D).

(iv) The administration structure (A) must not be adjacent to the dormitory (D).

(v) The administration structure (A) must not be on a hill.

(vi) The dormitory (D) must be on a hill or adjacent to the road.

(vii) All structures must be in different grid squares.

Here, adjacent means that the structures must share a grid edge, not just a corner.

Q1.1. Provide the domains of all variables after unary constraints have been applied.

Structure	Domain
A	{(1,1), (1,3), (2,2), (2,3)}
B	{(1,3), (2,3)}
C	{(1,1), (1,2), (1,3), (2,1), (2,2), (2,3)}
D	{(1,2), (1,3), (2,1), (2,3)}

Q1.2. Starting from the answer to Q1.1 (in which unary constraints are enforced), we are going to enforce arc consistency using the AC-3 algorithm. The initial queue of arcs is *{A → B, A → C, A → D, B → A, B → C, B → D, C → A, C → B, C → D, D → A, D → B, D → C}*. We start with enforcing arc consistency on *A → B*. Provide the domains of all variables after *A → B* is enforced.

Search for all non-hill squares that share an edge with at least one road adj square. For each a in Domain(A), if there is no adjacent b, we can remove a. (The rest are unchanged)

Structure	Domain
A	{(1,3), (2,2), (2,3)}
B	{(1,3), (2,3)}
C	{(1,1), (1,2), (1,3), (2,1), (2,2), (2,3)}
D	{(1,2), (1,3), (2,1), (2,3)}

Q1.3. Continuing from Q1.2, we keep enforcing arc consistency until we de-queue the arc $C \rightarrow B$ (i.e., remove $C \rightarrow B$ from the queue). Provide the domains of all variables after $C \rightarrow B$ is enforced.

The same principles from above are enforced, leaving only D unchanged now.

Structure	Domain
A	{(1,3), (2,2), (2,3)}
B	{(1,3), (2,3)}
C	{(1,2), (1,3), (2,2), (2,3)}
D	{(1,2), (1,3), (2,1), (2,3)}

Q1.4. What arcs got added to the queue while enforcing $C \rightarrow B$?

$A \rightarrow C$, $D \rightarrow C$ is enqueued.

Q1.5. Continuing from Q1.4, provide the domains of all variables after enforcing arc consistency until the queue is empty.

Structure	Domain
A	{(1,3), (2,2), (2,3)}
B	{(1,3), (2,3)}
C	{(1,2), (1,3), (2,2), (2,3)}
D	{(1,2), (1,3), (2,1), (2,3)}

Q1.6. If arc consistency had resulted in all domains having a single value left, we would have already found a solution. Similarly, if it had found that any domain had no values left, we would have already found that no solution exists. Unfortunately, this is not the case in our example (as you should have found in the previous part). To solve the problem, we need to start searching. Use the MRV (minimum remaining values) heuristic to choose which variable gets assigned next (breaking any ties alphabetically).

Which variable gets assigned next?

A=3, B=2, C=4, D=3. B has the minimum and gets assigned next (only 2 legal values remaining in domain)

Q1.7. Continuing from Q1.6, the variable you selected should have two values left in its domain. We will use the least-constraining value (LCV) heuristic to decide which value to assign before continuing with the search. To choose which value is the least-constraining value, enforce arc consistency for each value (on a scratch piece of paper). For each value, count the total number of values remaining over all variables. Which value has the largest number of values remaining (and therefore is the least constraining value)?

B = (1, 3) → inconsistent, 0 remaining values

LCV: B = (2, 3) → 4 remaining values (1 value each)

Q1.8. After assigning a variable, backtracking search with arc consistency enforces arc consis tency before proceeding to the next variable.

Provide the domains of all variables after assignment of the least-constraining value to the variable you selected and enforcing arc consistency. Note that you already did this computation to determine which value was the LCV.

Structure	Domain
A	{(1,3)}
B	{(2,3)}
C	{ (2,2)}
D	{(2,1)}

Q2. CSP Properties (True or False Answer) (18 points)

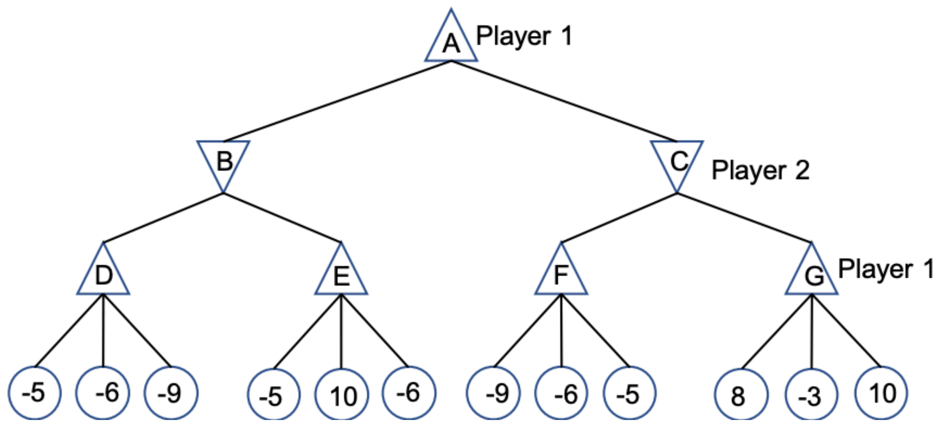
1. Even when using arc consistency, backtracking might be needed to solve a CSP.
    a. **True**
2. Even when using forward checking, backtracking might be needed to solve a CSP.
    a. **True**
3. When using backtracking search with the same rules to select unassigned variables and to order value assignments (in our case, usually Minimum Remaining Values and Least Constraining Value, with alphabetical tiebreaking), arc consistency will always give the same solution as forward checking, if the CSP has a solution.
    a. **False**
4. For a CSP with binary constraints that has no solution, some initial values may still pass arc consistency before any variable is assigned.
    a. **True**
5. If the CSP has no solution, it is guaranteed that enforcement of arc consistency results in at least one domain being empty.
    a. False
6. If the CSP has a solution, then after enforcing arc consistency, you can directly read off the solution from resulting domains.
    a. False
7. In general, to determine whether the CSP has a solution, enforcing arc consistency alone is not sufficient; backtracking may be required.
    a. True
8. If after a run of arc consistency during the backtracking search we end up with the filtered domains of *all* of the not yet assigned variables being empty, this means the CSP has no solution.

   a. **False**

9. If after a run of arc consistency during the backtracking search we end up with the filtered domains of *all* of the not yet assigned variables being empty, this means the search should backtrack because this particular branch in the search tree has no solution.
   a. **True**
10. If after a run of arc consistency during the backtracking search we end up with the filtered domain of *one* of the not yet assigned variables being empty, this means the CSP has no solution.
    a. **False**
11. If after a run of arc consistency during the backtracking search we end up with the filtered domains of all of the not yet assigned variables each having exactly one value left, this means we have found a solution.
    a. **True**
12. If after a run of arc consistency during the backtracking search we end up with the filtered domains of all of the not yet assigned variables each having more than one value left, this means we have found a whole space of solutions and we can just pick any combination of values still left in the domains and that will be a solution.
    a. **False**

Q3. Adversarial Search (30 points)

Q3.1. Search Algorithms. (20 points) Consider the zero-sum game tree shown below. Triangles that point up, such as at the top node (root), represent choices for the maximizing player; triangles that point down represent choices for the minimizing player. Outcome values for the maximizing player are listed for each leaf node, represented by the values in circles at the bottom of the tree.



1. (10 points) Assuming both players act optimally, carry out the minimax search algorithm. Enter the values for the letter nodes.

D	E	B	F	G	C	A
-5	10	-5	-5	10	-5	-5

2. (10 points) Assuming both players act optimally, use alpha-beta pruning to find the value of the root node. The search goes from left to right; when choosing which child to visit first, choose the left-most unvisited child.

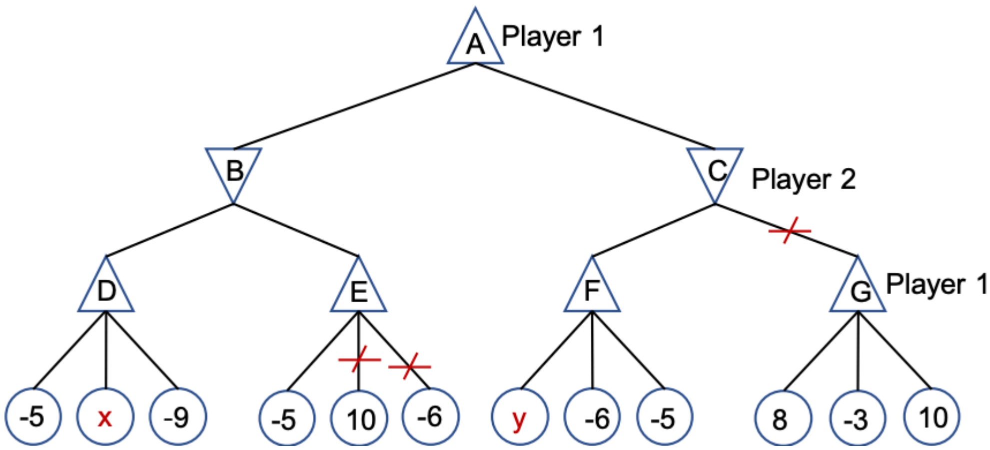
- Enter the values of the letter nodes.

D	E	B	F	G	C	A
-5	-5(10 & 6 pruned)	-5	-5	N/A, subtree	-5	-5

- Select the leaf nodes that don't get visited due to pruning.

When we visit node E → -5, we decide that leaf nodes 10 and 6 are not visited. Then after visiting all of F's child nodes, we decide that **8, -3** and **10** should not be visited.

Q3.2. Unknown Leaf Nodes. (10 points) Consider the following game tree with the same setting as Q3.1 except two of the leaves has an unknown payoff, *x* and *y*.



For what values of *x* and *y* such that the pruning shown in the figure will be achieved? The search goes from left to right; when choosing which child to visit first, choose the left-most un-visited child. Please specify your answer in one of the following forms:

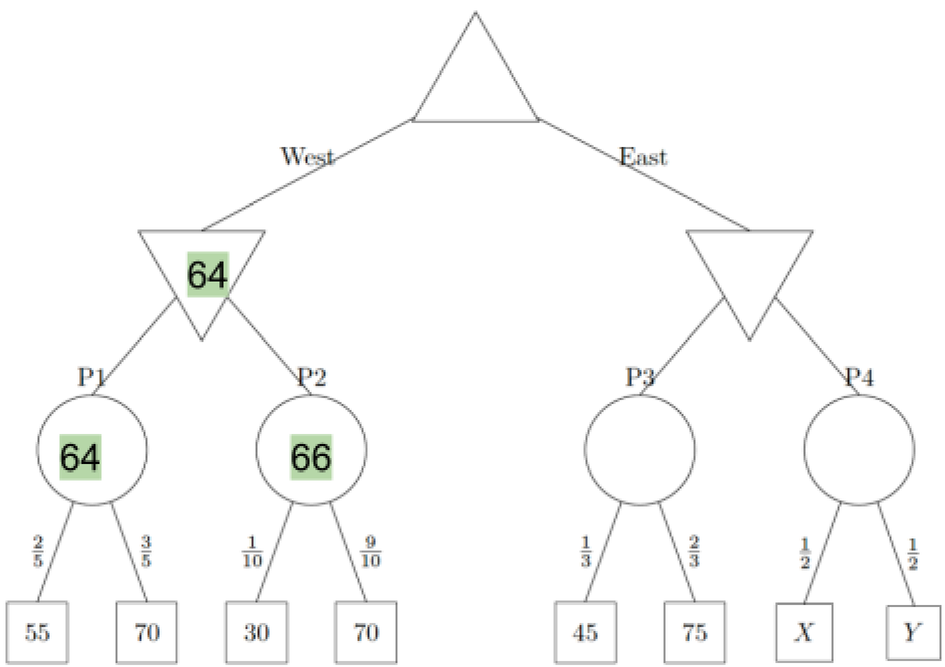
- Write All if *x* can take on all values
- Write None if *x* has no possible values
- Use an inequality in the form $x < \{value\}$, $x > \{value\}$, or $\{value1\} < x < \{value2\}$ to specify an interval of values. As an example, if you think *x* can take on all values larger than 16, you should enter $x > 16$.

x ≤ -5, y ≤ -5

Q4. Search with Uncertain Outcomes (20 points)

The following questions are completely unrelated to the above parts. Pacman is playing a tricky game. There are 4 portals to food dimensions. But, these portals are guarded by a ghost. Further more, neither Pacman nor the ghost know for sure how many pellets are behind each portal, though they know what options and probabilities there are for all but the last portal. Pacman moves first, either moving West or East. After which, the ghost can block 1 of the portals available. You have the following gametree. The maximizer node is Pacman. The minimizer nodes are ghosts and the portals are chance nodes with the probabilities indicated on the edges to the food. In the event of a tie, the left action is taken. Assume Pacman and the ghosts play optimally.

1. (10 points) Fill in values for the nodes that do not depend on X and Y.



2. (10 points) What conditions must X and Y satisfy for Pacman to move East? What about to definitely reach the P4? Keep in mind that X and Y denote numbers of food pellets and must be whole numbers: $X, Y \in \{0, 1, 2, 3, . . . \}$.
   a. Condition to move East: $X + Y \geq 129$
   b. Condition to move East & reach P4: $129 \leq X + Y \leq 130$

~~Q5. Possible Pruning (grads only) (25 points) (GRAD STUDENTS)~~

~~Assume we run $\alpha - \beta$ pruning, expanding successors from left to right, on a game with tree as shown in figure below. Which of the following statements are true?~~

1. ~~There exists an assignment of utilities to the terminal nodes such that no pruning will be achieved (shown in Figure (a)).~~
2. ~~There exists an assignment of utilities to the terminal nodes such that the pruning shown in Figure (b) will be achieved.~~
3. ~~There exists an assignment of utilities to the terminal nodes such that the pruning shown in Figure (c) will be achieved.~~
4. ~~There exists an assignment of utilities to the terminal nodes such that the pruning shown in Figure (d) will be achieved.~~
5. ~~There exists an assignment of utilities to the terminal nodes such that the pruning shown in Figure (e) will be achieved.~~
6. ~~None of the above.~~